

DevOps & Cloud

Interview Q&A Master Guide

27 Essential Questions with Detailed Answers

■ Cloud & AWS | ■ Scaling & Performance | ■ Networking & Infrastructure | ■ Kubernetes (EKS) | ■ DevOps Tools | ■ Linux & Troubleshooting | ■ Docker

Cloud & AWS

1. How will you design a highly available application on the cloud?

A highly available application uses redundancy at every layer to eliminate single points of failure. **Key components:** - **Multi-AZ deployment:** Run EC2/EKS nodes across at least 2–3 AZs using Auto Scaling Groups. - **Load Balancing:** Use ALB/NLB to distribute traffic; ALB supports path-based routing and health checks. - **Database HA:** RDS Multi-AZ for synchronous replication; failover is automatic (~60s). - **Stateless app tier:** Store sessions in ElastiCache (Redis), store files in S3 — not on local disk. - **Auto Scaling:** Scale EC2/ECS/EKS pods based on CPU, memory, or custom CloudWatch metrics. - **Health checks & Circuit breakers:** ALB deregisters unhealthy targets automatically. - **DNS failover:** Route 53 with health checks for multi-region routing. - **CDN:** CloudFront reduces origin load and adds edge caching for static assets. In my ApnaKart project: EKS across 3 AZs + ALB + RDS Multi-AZ + S3 + CloudFront achieved 99.9% uptime.

2. EC2 instance has a 50GB disk and 48GB is full — how to increase volume size without downtime?

You can expand an EBS volume live without stopping the instance. **Steps:** 1. Go to EC2 Console → Volumes → Modify Volume → increase size (e.g., 50GB → 100GB). 2. AWS resizes the EBS volume online (no reboot needed for Nitro instances). 3. SSH into the instance and extend the filesystem: **Check partition:** lsblk **Grow partition (if needed):** sudo growpart /dev/xvda 1 **Resize filesystem:** - For ext4: sudo resize2fs /dev/xvda1 - For xfs: sudo xfs_growfs / 4. Verify: df -h **Key points:** EBS modification is supported for gp2, gp3, io1, io2. You cannot shrink EBS volumes. Always snapshot before modifying as a precaution.

3. Difference between Multi-AZ and Read Replicas in AWS RDS?

Multi-AZ: - Purpose: High Availability (HA) and disaster recovery - Replication: Synchronous — data written to primary and standby simultaneously - Standby: Not accessible for reads; only used for automatic failover - Failover: Automatic in ~60–120 seconds via DNS CNAME flip - Use case: Production databases requiring zero data loss **Read Replicas:** - Purpose: Read scalability and performance - Replication: Asynchronous — slight lag possible - Replicas: Readable; can be in same AZ, different AZ, or different region - Failover: Manual promotion required - Use case: Offload read-heavy queries (reporting, analytics) **Can they coexist?** Yes — a Multi-AZ primary can also have Read Replicas attached.

4. How do you transfer data from an S3 bucket in one AWS account to another?

Method 1 — S3 Bucket Policy (most common): 1. On the source bucket, add a bucket policy allowing the destination account's IAM role/user to s3:GetObject and s3:ListBucket. 2. In the destination account, use aws s3 sync or aws s3 cp with the IAM role that has permission. **Method 2 — Cross-Account IAM Role:** 1. Create an IAM role in the source account with S3 read permissions. 2. Add a trust policy allowing the destination account to assume it. 3. From destination, use STS AssumeRole and then run aws s3 sync. **Method 3 — S3 Replication:** Enable Cross-Account S3 Replication on the source bucket — objects are automatically replicated to the destination bucket on an ongoing basis. **Key flag:** When syncing between accounts, use --acl bucket-owner-full-control so the destination account owns the copied objects.

Scaling & Performance

5. Your application suddenly receives high traffic — how would you handle it?

Immediate / Short-term: - Auto Scaling Group triggers scale-out based on CPU > 70% or ALB RequestCount metric - Kubernetes HPA scales pods automatically based on CPU/memory or custom metrics - CloudFront CDN absorbs static content requests at the edge, reducing origin load - ElastiCache (Redis/Memcached) serves cached DB responses — prevents DB bottleneck

Medium-term: - Read Replicas for RDS to offload read queries - SQS queue to decouple and buffer bursts — async processing via Lambda or workers - Implement rate limiting at API Gateway or ALB level **Architectural (long-term):** - Move to microservices so only the bottlenecked service scales - Implement circuit breakers to prevent cascade failures - Use predictive scaling based on historical traffic patterns In ApnaKart: HPA + ALB + CloudFront handled traffic spikes with 99.9% uptime.

6. What is Auto Scaling? Explain min, max, and desired capacity.

Auto Scaling automatically adjusts the number of EC2 instances (or ECS tasks/EKS nodes) in response to demand. **Three capacity settings in an Auto Scaling Group (ASG):** - **Minimum capacity:** The floor. ASG will never go below this number, even at zero load. Ensures baseline availability. (e.g., min=2 keeps 2 instances always running) - **Maximum capacity:** The ceiling. ASG will never exceed this, preventing runaway scaling costs. (e.g., max=10) - **Desired capacity:** The target number of instances right now. ASG actively maintains this count. Scaling policies change the desired value, which then triggers launch/terminate events. (e.g., desired=4 means 4 instances running currently) **Scaling triggers:** CloudWatch alarms on CPU utilization, memory (custom metric), ALB request count, or scheduled scaling for predictable patterns. **Scaling policies types:** Simple scaling, Step scaling, Target Tracking (most recommended — e.g., "keep CPU at 60%").

7. How would you design a system for high availability across multiple regions?

Architecture layers: - **DNS layer:** Route 53 with latency-based or failover routing. Health checks automatically reroute traffic if a region is unhealthy. - **Compute:** Deploy EKS/EC2 clusters in 2+ regions (e.g., ap-south-1 + us-east-1). Use Active-Active or Active-Passive setup. - **Data layer:** RDS Global Database (Aurora) for sub-1-second replication across regions. DynamoDB Global Tables for multi-region writes. - **Storage:** S3 Cross-Region Replication (CRR) for static assets and backups. - **CDN:** CloudFront with origins in multiple regions — automatically routes to nearest healthy origin. **Active-Active vs Active-Passive:** - Active-Active: Both regions serve traffic (better performance, complex data sync) - Active-Passive: One region is standby, only takes over on failure (simpler, slight delay) **RTO/RPO targets:** Always define these upfront — they drive architecture decisions on

replication lag tolerance.

Networking & Infrastructure

8. In a VPC with two subnets, how do you identify which is public and which is private?

The definitive indicator — Route Table: - **Public subnet:** Its route table has a route to an Internet Gateway (IGW): 0.0.0.0/0 → igw-xxxxxxx - **Private subnet:** Its route table routes outbound traffic through a NAT Gateway: 0.0.0.0/0 → nat-xxxxxxx (or has no internet route at all) **Secondary indicators:** - Public subnet: "Auto-assign public IPv4 address" is enabled - Private subnet: Instances have only private IPs; no direct internet access - NAT Gateway itself lives in the public subnet (it needs an Elastic IP + IGW access) **Quick check via CLI:** `aws ec2 describe-route-tables --filters "Name=association.subnet-id,Values=subnet-xxxx"` Look for the GatewayId — if it starts with igw-, it's public.

9. Explain Docker networking.

Docker provides several network drivers: **bridge (default):** Containers on the same host communicate via a virtual bridge network. Each container gets an internal IP. Use `--network bridge` or just run without specifying. **host:** Container shares the host's network namespace. No network isolation — container uses host's IP directly. Better performance, less security. **none:** No network interface. Completely isolated container. **overlay:** Used in Docker Swarm/multi-host setups. Enables containers on different hosts to communicate as if on the same network. Uses VXLAN encapsulation. **macvlan:** Assigns a MAC address to the container, making it appear as a physical device on the network. **Key commands:** - `docker network ls` — list networks - `docker network create mynet` — create custom bridge network - `docker network connect mynet container1` — attach container to network - `docker inspect container` — see IP and network info **Best practice:** Use custom bridge networks (not default bridge) — they support DNS resolution by container name.

10. CMD vs ENTRYPOINT in Docker?

Both define what runs when a container starts, but they behave differently. **CMD:** - Provides default command/arguments - Easily overridden at runtime: `docker run myimage echo hello` — replaces CMD entirely - Use when you want a flexible default **ENTRYPOINT:** - Defines the fixed executable that always runs - Runtime arguments append to it, not replace it - `docker run myimage --port 8080` — passes `--port 8080` as args to ENTRYPOINT **Combined use (best practice):** `ENTRYPOINT ["python", "app.py"]` `CMD ["--port", "8080"]` Here, CMD provides default args to ENTRYPOINT. Override just the args at runtime without changing the executable. **Shell vs Exec form:** - Exec form (recommended): `["python", "app.py"]` — no shell, signals propagate correctly to PID 1 - Shell form: `python app.py` — runs via `/bin/sh -c`, signal handling issues, avoid for ENTRYPOINT

Kubernetes (K8s)

11. What are common Kubernetes security measures in EKS?

Identity & Access: - IRSA (IAM Roles for Service Accounts): Pod-level AWS permissions via OIDC — no hardcoded credentials - RBAC: Restrict what each user/service account can do (Roles + RoleBindings) - `aws-auth ConfigMap`: Controls which IAM users/roles map to Kubernetes RBAC groups **Network Security:** - Network Policies: Restrict pod-to-pod traffic (deny-all default, allow only needed) -

Private API server endpoint: Restrict kubectl access to VPN/bastion only - Security Groups for Pods: Fine-grained SG rules at pod level (EKS supports this) **Image Security:** - Trivy image scanning in CI pipeline — block images with CRITICAL CVEs - ECR with image scanning enabled + signed images - Restrict public image pulls via admission controllers **Runtime Security:** - Pod Security Standards (restricted/baseline profiles) - Secrets encryption at rest using AWS KMS with envelope encryption - Falco for runtime threat detection **Audit & Compliance:** - Enable EKS control plane logging (audit, authenticator, api) to CloudWatch - Use OPA/Gatekeeper for policy enforcement

12. What are Kubernetes Services and how are they used?

A Service provides a stable network endpoint to a set of pods (selected by labels), since pod IPs are ephemeral. **Service types:** **ClusterIP (default):** Internal-only. Accessible within the cluster. Used for pod-to-pod communication. **NodePort:** Exposes service on a static port on every node's IP (30000–32767). Accessible externally via NodeIP:NodePort. Not for production. **LoadBalancer:** Provisions a cloud load balancer (AWS ALB/NLB). External traffic entry point for production. In EKS, use AWS Load Balancer Controller for ALB. **ExternalName:** Maps a service to an external DNS name (e.g., an RDS endpoint). No proxying, just DNS CNAME. **How it works internally:** kube-proxy sets iptables/IPVS rules on each node. Service has a ClusterIP and DNS name (my-svc.namespace.svc.cluster.local). Traffic to ClusterIP is load-balanced across healthy pod endpoints via Endpoints object. **In ApnaKart:** Each microservice exposed via ClusterIP internally; gateway exposed via LoadBalancer (ALB).

13. What is CrashLoopBackOff? How do you troubleshoot it?

CrashLoopBackOff means the container is repeatedly crashing after start. Kubernetes keeps restarting it with exponential backoff delay (10s, 20s, 40s... up to 5 min). **Root causes:** - Application error/exception on startup (wrong config, missing env var, bad code) - Missing or incorrect ConfigMap/Secret mounted - OOMKilled — container exceeds memory limit - Liveness probe failing too aggressively - Wrong CMD/ENTRYPOINT in Docker image **Troubleshooting steps:** 1. kubectl describe pod — check Events section for exit codes and reasons 2. kubectl logs --previous — logs from the crashed container 3. kubectl logs -c — if multi-container pod 4. Check exit code: Exit code 1 = app error, 137 = OOMKilled, 126/127 = command not found 5. kubectl get pod -o yaml — check resource limits, env vars, volume mounts 6. Temporarily override CMD: kubectl run debug --image=myimage --sleep 3600 to inspect inside **Fix examples:** Increase memory limits for OOM, fix env var injection, relax liveness probe initialDelaySeconds.

14. Pod is stuck in Pending state — how do you debug?

Pending means the scheduler cannot place the pod on any node yet. **Step 1 — kubectl describe pod** Look at the Events section — it tells you exactly why scheduling failed. **Common causes and fixes:** - **Insufficient CPU/Memory:** No node has enough resources. Fix: scale up nodes, reduce pod requests, or enable Cluster Autoscaler. - **No nodes match node selector/affinity:** Pod requires a label the nodes don't have. Fix: check nodeSelector, nodeAffinity, and node labels (kubectl get nodes --show-labels). - **Taint/Toleration mismatch:** Node has a taint the pod doesn't tolerate. Fix: add toleration to pod spec or remove taint. - **PVC not bound:** Persistent Volume Claim is pending (no matching PV). Fix: check PVC status, StorageClass, and EBS availability zone alignment. - **Image pull issue:** If pod reaches ContainerCreating then fails — check imagePullSecrets and ECR permissions. **Useful commands:** kubectl get events --sort-by=.lastTimestamp kubectl describe node — check Allocatable vs Requests

15. Difference between rolling updates and rollback?

Rolling Update: Kubernetes replaces pods incrementally — new pods come up, old ones go down gradually. Zero downtime. Controlled by: - maxSurge: How many extra pods can exist above desired count during update (e.g., 25%) - maxUnavailable: How many pods can be down during update (e.g., 25%) Trigger: `kubectl set image deployment/myapp app=myimage:v2` Or update image in deployment YAML and apply. **Rollback:** Reverts to a previous ReplicaSet. Kubernetes keeps rollout history (default: 10 revisions). Commands: - `kubectl rollout status deployment/myapp` — watch update progress - `kubectl rollout history deployment/myapp` — see revision history - `kubectl rollout undo deployment/myapp` — rollback to previous version - `kubectl rollout undo deployment/myapp --to-revision=3` — rollback to specific revision **Key point:** A rollback is itself a rolling update — it doesn't kill all pods at once. The old ReplicaSet is scaled back up gradually.

16. Explain Blue-Green vs Canary deployments.

Blue-Green Deployment: - Two identical environments: Blue (live) and Green (new version) - Deploy new version to Green, run tests, then switch traffic 100% from Blue to Green instantly - Rollback = flip traffic back to Blue (still running) - **Pros:** Instant switch, easy rollback, zero downtime - **Cons:** Requires 2x infrastructure cost, no gradual validation **Canary Deployment:** - Route a small % of traffic to the new version (e.g., 5%), rest goes to stable - Monitor metrics (error rate, latency, business KPIs) - Gradually increase traffic to new version if healthy; rollback if issues arise - **Pros:** Real user validation, risk limited to small %, catch issues early - **Cons:** More complex traffic management, longer rollout window **In Kubernetes:** - Blue-Green: Switch Service selector between two Deployments - Canary: Use Argo Rollouts, Istio weighted routing, or AWS ALB weighted target groups **In ApnaKart:** Used rolling updates for standard deploys; Canary pattern was applied via ALB weighted target groups for major service changes.

17. How do you provide read-only access to a developer in Kubernetes (RBAC)?

RBAC in Kubernetes has 4 objects: Role, ClusterRole, RoleBinding, ClusterRoleBinding. **Step 1 — Create a Role (namespace-scoped read-only):** `apiVersion: rbac.authorization.k8s.io/v1 kind: Role metadata: namespace: production name: read-only rules: - apiGroups: [""] resources: ["pods", "services", "configmaps", "events"] verbs: ["get", "list", "watch"] - apiGroups: ["apps"] resources: ["deployments", "replicasets"] verbs: ["get", "list", "watch"]` **Step 2 — Bind the Role to the developer:** `apiVersion: rbac.authorization.k8s.io/v1 kind: RoleBinding metadata: name: read-only-binding namespace: production subjects: - kind: User name: dev-sanket apiGroup: rbac.authorization.k8s.io roleRef: kind: Role name: read-only apiGroup: rbac.authorization.k8s.io` **For cluster-wide read access:** Use ClusterRole + ClusterRoleBinding instead. **In EKS:** Map IAM user to the Kubernetes user via `aws-auth ConfigMap` first.

DevOps Tools & Practices

18. How do you use Grafana? How did you integrate it in your project?

Grafana is a visualization and dashboarding tool that queries metrics from data sources like Prometheus, Datadog, CloudWatch. **Core concepts:** - **Data Sources:** Configure Prometheus, Datadog, CloudWatch etc. as data sources - **Dashboards:** Build panels with PromQL/metrics queries visualized as graphs, gauges, tables - **Alerts:** Set thresholds — notify via Slack, PagerDuty, email when metric breaches **In ApnaKart project:** - Prometheus scraped metrics from all 10 microservices (Spring Boot Actuator endpoints exposed `/metrics`) - Grafana connected to Prometheus as data source - Built

dashboards tracking: JVM heap, HTTP request rate, error rate (5xx), pod CPU/memory, RDS connections - Set alert rules: CPU > 80% for 5 min triggered Slack alert; Error rate > 1% triggered PagerDuty - Datadog APM was deployed as a DaemonSet for distributed tracing — Grafana also visualized Datadog metrics via Datadog data source plugin **Key panels used:** Time series for latency, Stat panels for uptime %, Heatmaps for request distribution.

19. What tools have you used for Kubernetes (self-managed vs cloud)?

Cloud-managed (EKS) — Used in ApnaKart: - AWS EKS: Control plane managed by AWS; worker nodes are EC2 in Auto Scaling Groups - eksctl: Cluster creation and node group management - AWS Load Balancer Controller: Provision ALB/NLB from Kubernetes Ingress/Service - IRSA: IAM roles for service accounts via OIDC — secure pod-level AWS access - EKS Managed Node Groups: AWS handles node patching and scaling **Local/self-managed tools used:** - minikube: Local single-node cluster for development and testing - kubectl: Primary CLI for all cluster operations - Helm: Package manager for deploying charts (Prometheus, Grafana, ingress-nginx) - Kustomize: Environment-specific overlay management (dev/test/uat/prod configs without duplicating YAML) **Observability tools on K8s:** - Prometheus + Grafana: Metrics stack deployed via Helm - Datadog Agent as DaemonSet: APM and infrastructure monitoring - Trivy: Image scanning in CI before push to ECR

20. What do you do in Terraform? How do you write and manage infra code?

Terraform is an IaC tool that provisions and manages cloud infrastructure declaratively using HCL. **In ApnaKart — I wrote Terraform modules for:** - VPC module: 3 public + 3 private subnets across 3 AZs, IGW, NAT Gateway, route tables - EKS module: Cluster + managed node groups + OIDC provider for IRSA - RDS module: Multi-AZ PostgreSQL with subnet groups, parameter groups, security groups - S3 + CloudFront module: Static asset hosting with OAC (Origin Access Control) - Route53 module: Hosted zone, A records, health checks **Code organization:** modules/ — reusable modules (vpc, eks, rds) environments/dev/ — dev-specific tfvars and backend config environments/prod/ — prod-specific config Each environment has its own state file in S3 (backend "s3") with DynamoDB state locking. **Workflow:** terraform init → terraform plan → terraform apply In CI/CD: Terraform plan output posted as PR comment; apply runs after merge. **Key practices:** Remote state in S3 + DynamoDB locking, workspaces for env separation, terraform.tfvars for secrets (never hardcoded), output values shared between modules via data sources.

21. How would you build a 3-tier architecture using Infrastructure as Code?

3-Tier Architecture: Presentation → Application → Data Tier 1 — Presentation (Web/CDN): - S3 bucket for static React/frontend assets - CloudFront distribution in front of S3 - Route53 DNS pointing to CloudFront **Tier 2 — Application (Compute):** - VPC with public subnets (ALB) and private subnets (app servers/EKS nodes) - ALB in public subnets with Target Groups pointing to EKS pods or EC2 - EKS cluster in private subnets — pods run application microservices - Auto Scaling for EC2 worker nodes **Tier 3 — Data (Database):** - RDS Multi-AZ in isolated private subnets (no route to internet) - Security Group: Only allow inbound 5432/3306 from application tier SG - Secrets: DB credentials stored in AWS Secrets Manager, injected via External Secrets Operator **Terraform structure:** module "vpc" → module "eks" → module "rds" → module "alb" → module "cloudfront" Modules depend on each other via outputs (vpc_id, subnet_ids, etc.) This is exactly the structure I implemented in ApnaKart — 149+ Terraform files across 6 modules.

22. How to check open ports in Linux?

Several tools, each useful in different situations: **ss (modern, preferred)**: `ss -tulnp -t: TCP, -u: UDP, -l: listening, -n: numeric ports, -p: show process` **netstat (older but widely available)**: `netstat -tulnp` **lsof (process-level detail)**: `lsof -i :8080` — show what's using port 8080 `lsof -i -P -n | grep LISTEN` **nmap (scan from outside)**: `nmap -sV localhost` — scan all open ports with service version `nmap -p 22,80,443` — scan specific ports **firewall check (what's allowed)**: `iptables -L -n` (traditional) `ufw status` (Ubuntu) `firewall-cmd --list-ports` (CentOS/RHEL) **Quick one-liner to check if specific port is open**: `nc -zv hostname 443` — returns success/failure

23. How to find large files in Linux?

find command (most precise): `find / -type f -size +100M 2>/dev/null` `find /var -type f -size +500M -exec ls -lh {} \;` **du — disk usage by directory**: `du -sh /* 2>/dev/null | sort -rh | head -20` `du -ah /var/log | sort -rh | head -10` **df — check which filesystem is full first**: `df -h` — shows disk usage per mount point **ncdu (interactive, highly recommended)**: `ncdu /` — interactive tree view, navigate with arrow keys, instantly find large dirs **Common places to check for bloat**: `- /var/log` — application and system logs `- /var/lib/docker` — Docker images, containers, volumes `- /tmp` — temp files `- ~/.m2` or `~/.gradle` — Maven/Gradle caches (on build servers) `- Core dumps`: `find / -name "core" -type f` **Docker-specific**: `docker system df` — shows space used by images, containers, volumes `docker system prune -a` — clean unused resources

24. How to find files modified in the last 30 minutes?

Using find with -mmin: `find /path -type f -mmin -30` `-mmin -30`: modified less than 30 minutes ago `-mmin +30`: modified more than 30 minutes ago **Examples**: `find /etc -type f -mmin -30` — config files changed recently `find /var/log -type f -mmin -30 -exec ls -lh {} \;` — log files with sizes `find . -mmin -30 -name "*.log"` — recently modified log files in current dir **Find files modified today**: `find / -type f -newer /tmp/ref_file` (where `ref_file` was touched at start of day) OR: `find / -type f -mtime 0` (modified within last 24h) **Using stat for specific file**: `stat filename` — shows `atime`, `mtime`, `ctime` **Monitoring in real-time**: `inotifywait -m /path` — monitor directory for changes in real-time (requires `inotify-tools`)

25. How do you troubleshoot disk space issues?

Step 1 — Identify which partition is full: `df -h` — check usage % per mount **Step 2 — Find what's consuming space**: `du -sh /* 2>/dev/null | sort -rh | head -20` Drill down: `du -sh /var/* | sort -rh` **Step 3 — Common culprits and fixes**: `- /var/log full`: `journalctl --vacuum-size=500M` or `logrotate`, truncate old logs `- Docker`: `docker system prune -a` (removes unused images, stopped containers) `- Old kernels (Ubuntu)`: `apt autoremove --purge` `- Temp files`: `rm -rf /tmp/*` (carefully) `- Core dumps`: `find / -name "core" -delete` `- Large log files`: `> /var/log/syslog` (truncate without deleting if process has file open) **Step 4 — If a deleted file is still holding space**: `lsof | grep deleted` — find processes holding deleted files open Restart that process to release the space **Step 5 — Expand if needed**: AWS EBS: Modify volume size + grow partition + `resize2fs/xfs_growfs`

Docker

26. What is multi-stage Docker build?

Multi-stage builds use multiple `FROM` statements in one Dockerfile, allowing you to use a heavy build image and copy only the compiled artifacts into a lean final image. **Example — Java Spring Boot**: #

Stage 1: Build FROM maven:3.9-eclipse-temurin-17 AS builder WORKDIR /app COPY pom.xml . RUN mvn dependency:go-offline COPY src ./src RUN mvn package -DskipTests # Stage 2: Runtime FROM eclipse-temurin:17-jre-alpine WORKDIR /app COPY --from=builder /app/target/*.jar app.jar EXPOSE 8080 ENTRYPOINT ["java", "-jar", "app.jar"] **Why it matters:** - Build stage has Maven, JDK, source code, dependencies (~600MB+) - Final image has only JRE + JAR (~150MB) - No build tools, source code, or intermediate files in production image - Smaller attack surface, faster pulls, lower ECR storage costs **In ApnaKart:** Used multi-stage builds for all 10 Spring Boot microservices — achieved ~60% image size reduction. Trivy scanning in CI ran against the final stage image.

27. Why avoid Alpine images sometimes? What are alternatives?

Alpine is attractive (5MB base) but has real gotchas: **Reasons to avoid Alpine:** - **musl libc vs glibc:** Alpine uses musl libc instead of glibc. Some compiled binaries (especially JVM, Python native extensions, glibc-linked C libraries) don't work correctly or crash. - **Missing debugging tools:** No bash (only sh), no curl/wget by default — painful to debug in production - **DNS resolution bugs:** Alpine's musl has known DNS timeout issues in some containerized environments (particularly with ndots settings in Kubernetes) - **Python packages:** Some pip packages with C extensions (numpy, pandas) need to be compiled from source on Alpine — slow builds, potential incompatibilities **Better alternatives:** - **Distroless (Google):** No shell, no package manager — smallest attack surface. Ideal for production Go/Java apps. (gcr.io/distroless/java17) - **Debian Slim:** debian:bookworm-slim — uses glibc, ~80MB, most compatible - **Ubuntu Minimal:** ubuntu:22.04 — familiar, good compatibility, ~70MB - **Eclipse Temurin JRE:** eclipse-temurin:17-jre — optimized for Java, glibc-based **Rule of thumb:** Alpine is fine for Go binaries (statically compiled). For Java, Python, or anything with native libs — use Distroless or Debian Slim.
